

# PI-Fed: Continual Federated Learning With Parameter-Level Importance Aggregation

Lang Yu<sup>1</sup>, Graduate Student Member, IEEE, Lina Ge<sup>2</sup>, Guanghui Wang<sup>2</sup>, Member, IEEE, Jianghao Yin, Qin Chen<sup>3</sup>, Jie Zhou<sup>3</sup>, and Liang He<sup>3</sup>

**Abstract**—Federated Learning (FL) has drawn much attention for distributed system over the Internet of Things (IoT), since it enables collaborative machine learning on heterogeneous devices while resolves concerns about privacy leakage. Due to the catastrophic forgetting (CF) phenomenon of optimization methods, existing FL approaches are restricted to single task learning and typically assume that data from all nodes are simultaneously available during training. However, in practical IoT scenarios, the data preparation from nodes may be asynchronous, and different tasks require incremental training. To address the issues, we propose a continual FL (CFL) framework with parameter-level importance aggregation (PI-Fed), which supports collaborative task-incremental learning with privacy preservation. Specifically, PI-Fed evaluates the importance of each parameter in the global model to all history tasks, which is computed locally and aggregated at the center server. Then the server performs softmasking on the averaged gradient collected from local clients based on the parameter importance. By minimizing the change on important parameters, PI-Fed effectively overcomes CF and also achieves high efficiency without experience replay. Extensive experiments on 4 benchmarks with at most 20 sequential tasks demonstrate that our proposed PI-Fed significantly outperforms traditional FL baselines (FedAvg, FedNova, and SCAFFOLD).

**Index Terms**—Continual learning (CL), federated learning (FL), first-order optimization, parameter importance.

## I. INTRODUCTION

THE FUSION of Internet of Things (IoT) [1] and machine learning [2] represents a novel paradigm for collaborative training and data-driven decision-making. With the compliance of data security and privacy regulations, the combination of IoT and federated learning (FL) [3], [4] demonstrates potential

to revolutionize data processing with machine learning in distributed environments. From the fundamental FedAvg [5] initially introduced by McMahan in 2016, to asynchronous FL (AFL) [6] placing more emphasis on practical operations in the real world (heterogeneous devices, aggregation efficiency and low-resource utilization), FL has presented a wide range of merits to the IoT applications [7].

Most existing FL approaches are restricted to single task learning and typically assume that data from all nodes are simultaneously available during training. While in real-world FL scenarios, the data preparation and the participation of different clients may be asynchronous. Consequently, the sequentially trained global model tends to forget previously learned tasks, which is also known as the catastrophic forgetting (CF) phenomenon. A promising solution is to adopt continual learning (CL) strategies within the FL framework, which enables knowledge accumulation over time without CF on previously learned tasks [8]. Prior works on CL can be roughly categorized into three groups, namely, memory replay [9], [10], [11], weight regularization [12], [13], [14], and parameter-isolation [15], [16], [17] methods, whereas CL is typically designed for centralized model training, and it's challenging to directly integrate aforementioned CL methods into FL frameworks under distributed IoT environment.

Some researchers attempt to narrow the gap between CL and FL, which could mitigate CF during privacy-preserving training, and meanwhile achieve knowledge transfer (KT), meaning that leveraging previously learned knowledge to improve the performance on new tasks. Shoham et al. [18] adapted EWC [12] to FL, which penalizes the change of parameters that are most informative for previous tasks. Usmanova et al. [19] proposed a distillation-based method to deal with CF in FL, which regards the server as a teacher network that sends prior knowledge to each student client model. Ma et al. [20] further employed knowledge distillation based on surrogate data sets to mitigate forgetting both at the server-side and client-side. Although the above methods partially address the issue of continual FL (CFL) for IoT, they still compromise on clients' privacy or exhibit subpar performance, due to their singular reliance on either simple data replay or general gradient regularization, rather than fine-grained parameter-level analysis and calibration. Additionally, these methods often incur substantial time and storage overheads, which underscores the necessity for an effective and efficient CFL algorithm.

Manuscript received 15 July 2024; accepted 4 August 2024. Date of publication 7 August 2024; date of current version 7 November 2024. This work was supported in part by the National Science and Technology Major Project under Grant 2021ZD0114002; in part by the National Natural Science Foundation of China (NSFC) under Grant 62307028 and Grant 92367302; in part by the Science and Technology Commission of Shanghai Municipality under Grant 22511105901, Grant 21511100402, and Grant 23ZR1441800; and in part by the Key Technology Research and Development Project of Henan Province under Grant 242102210139 and Grant 242102211097. (Corresponding author: Lina Ge.)

Lang Yu, Jianghao Yin, Qin Chen, Jie Zhou, and Liang He are with the School of Computer Science and Technology, East China Normal University, Shanghai 200333, China (e-mail: llyu@stu.ecnu.edu.cn; jhyin@stu.ecnu.edu.cn; qchen@cs.ecnu.edu.cn; jzhou@cs.ecnu.edu.cn; lhe@cs.ecnu.edu.cn).

Lina Ge is with the School of Software, Henan Finance University, Zhengzhou 450000, China (e-mail: gelina@hafu.edu.cn).

Guanghui Wang is with the School of Software, Henan University, Zhengzhou 450000, China (e-mail: gwang@vip.henu.edu.cn).

Digital Object Identifier 10.1109/JIOT.2024.3440029

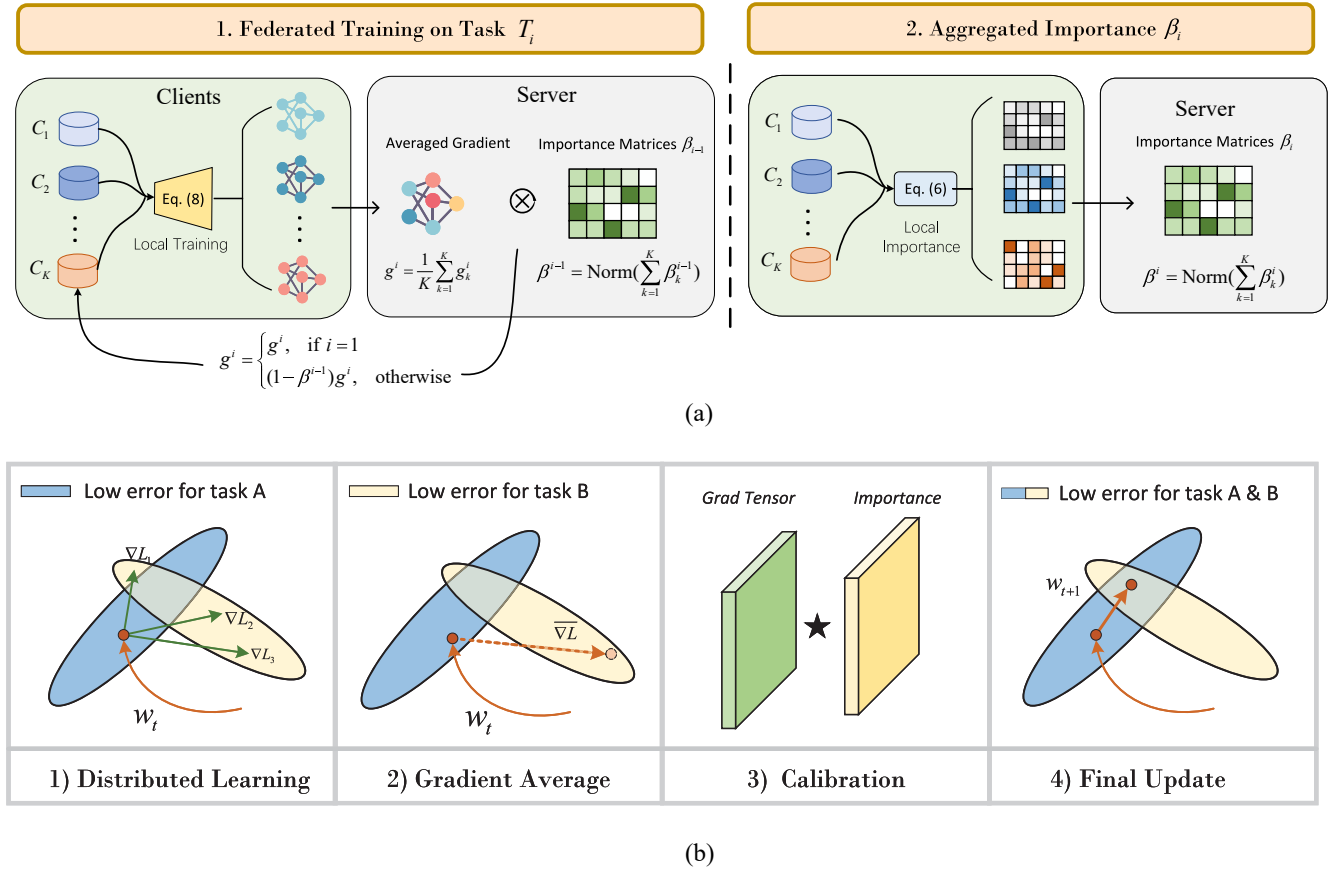


Fig. 1. (a) Entire framework of our proposed PI-Fed, which is composed of two stages, namely, FedAvg with soft-masking and federated importance aggregation. (b) Visualization of gradient soft-masking, which utilizes the Hadamard product of the averaged gradient and normalized importance matrix as the calibrated gradient for broadcasting.

In this article, we propose a novel framework, CFL framework with parameter-level importance aggregation (PI-Fed) to enable CFL in the multitask and life-long IoT environments, which introduces a soft-masking strategy to the center server based on the parameter importance to regularize the gradient during the weight average. Specifically, the importance of parameters will be evaluated at the end of learning of each task, which identifies the coordinates of parameters in the network that are the most informative for previous tasks. As shown in Fig. 1(a) PI-Fed uses the importance score of each parameter as a soft-mask to constrain the gradient accumulation in the center server to ensure those important parameters to previous tasks have minimum changes in learning a new task. This method could achieve great performance on CFL tasks with high efficiency, which set the stages for future studies and exploration.

The key contributions<sup>1</sup> of this work can be summarized as follows.

- 1) We extend CL to the FL and formulate the CFL, which enables knowledge accumulation without CF on sequential tasks in the IoT environment.
- 2) We propose a novel CFL framework PI-Fed, which adopts soft-masking strategy based on the aggregated parameter importance to calibrate the server-side global

gradient. PI-Fed effectively enables knowledge accumulation and mitigate the CF with high efficiency.

- 3) Experimental results on CIFAR100-n, TinyImageNet-n, ImageNet100, and F-CelebA-n demonstrate that PI-Fed outperforms both the classic CL methods and other FL baseline. The follow-up analysis and discussion set the stage for future research and improvement on the adaptive CFL.

## II. RELATED WORK

### A. Asynchronous Federated Learning

Following the classic synchronous FL paradigm (Initialization, Local model training and Global model aggregation), researchers have presented innovative frameworks for expanded array of applications [21]. However, with the significant development of heterogeneous devices and data sets, several drawbacks of synchronous FL become apparent [6], such as device unreliability, data over-fitting, and aggregation overload. This indicates a need of developing AFL algorithms under the IoT environment [22].

One frequently stated problem is how to optimize across heterogeneous devices with high efficiency. Existing studies encapsulates several axes, namely, encompassing node selection [23], [24], [25], weighted aggregation [26], [27], [28], gradient compressing [29], [30], cluster FL [31], [32],

<sup>1</sup>Source code is available at <https://github.com/BruthYU/PI-Fed>

and model splitting [33]. Recently, another significant aspect of AFL focuses on the asynchrony of training progress [24], [34], [35], where the clients can communicate with the server at arbitrary time intervals. Avdiukhin and Kasiviswanathan [36] investigated the performance of an asynchronous version of local stochastic gradient descent (SGD) optimizer. FedBuff [37] further proposes a novel buffered asynchronous aggregation method that is agnostic to the choice of optimizer, and combines the best properties of synchronous and AFL. Our PI-Fed also contributes to the study on asynchronous training progress, while the major difference compared to previous work lies in the requirement for CL, where the method needs to achieve KT for new tasks and meanwhile mitigate CF without replay on old data.

### B. Continual Learning

CL aims to develop algorithms that enable neural networks learning new knowledge while maintaining the information learned from previous training data. Commonly used CL strategies could be roughly grouped into three categories: 1) memory replay methods [9], [10], [11], [38] keep a part of the previous training data in a memory buffer for future retraining; 2) regularization-based methods [12], [13], [14] restrict the change of weights to alleviate the CF; and 3) parameter-isolation methods [15], [16], [17] use masking to learn subnetworks for each task.

Most CL methods are implemented based on the centralized training, meaning that the learning process is controlled by a single server and the data set is required to be public without privacy preservation. Relatively fewer works have studied the intersection of continual and FL, including aforementioned studies, CFL can be categorized into knowledge sharing and gradient regularization methods. In the former group [18], [39], [40], all clients provide a buffer of public data for replay and continually learn from each other as the training proceeds. While for the regularization-based methods [19], [20], [41], the gradient on the global model at the center server will be calibrated before the averaging operation, according to the prior knowledge determined by the weight learned from the initial task or distillation loss from the teacher network. However, knowledge sharing compromises on the clients' privacy and rough gradient regularization on the entire network may lead to subpar performance. Our proposed PI-Fed belongs to the regularization-based methods, but the fine-grained parameter-level regularization on gradient could keep more previously learned knowledge, and achieve better KT by fixing the important parameters.

### C. First-Order Optimization Methods

Optimization encompasses a central role in deep learning [42]. The optimization techniques involving the first-order information of the gradient are most widely used in FL. Polyak [43] introduced the momentum method to accelerate the learning process of first-order gradient-based optimization. Later Adagrad [44] algorithm, respectively, adjusts the learning rate of all trainable parameters by scaling them inversely proportional to the root of mean squared historical gradient.

TABLE I  
DEFINITION OF NOTATIONS

| Symbols            | Description                                          |
|--------------------|------------------------------------------------------|
| $\theta$           | Weights of the Global Model                          |
| $K$                | Number of Clients in One Task                        |
| $k$                | Client $k$ with the $k$ -th Data Sub-Collection      |
| $n_k$              | Number of the samples in $k$ -th Data Sub-Collection |
| $n_a$              | Number of the samples in entire Data Collection      |
| $T_i^k$            | $k$ -th Data Collection of $i$ -th Task              |
| $L^{i,\tau}$       | Loss for Task $\tau$ during Task $i$                 |
| $f$                | The Global Model                                     |
| $X_k$              | Samples of the $k$ -th Data Sub-Collection           |
| $Y_k$              | Labels of the $k$ -th Data Sub-Collection            |
| $\beta_k^{i,\tau}$ | Local Importance for Task $\tau$ during Task $i$     |
| $\beta_k^i$        | Local Maximum Importance Value                       |
| $g_k^i$            | Gradient from Client $k$ in the Local Training Phase |
| $g^i$              | Averaged Gradient at Server during Task $i$          |
| $g^i_\star$        | Masked Gradient at Server                            |

RMSProp [45] method modifies the Adagrad by changing the accumulated square gradient into an exponentially weighted moving average. The well-known Adam [46] algorithm aligns along the works of RMSProp and further build up with a few distinctions, which is invariant to the diagonal rescaling of the gradients. Despite the broad adoption in deep learning applications, these optimization methods are designed for single task learning, which can not be directly employed in the scenario of CFL. To tackle this problem, our proposed PI-Fed introduces the parameter-level importance for each task to the optimization process, which is used to calibrate the final update on each parameter accumulated by the optimization methods.

## III. PROBLEM FORMULATION

We first take a review of the vanilla FL, which assumes that a center server  $S$  and  $K$  clients to train a model  $\theta$  on task  $T$  through  $R$  rounds of communications. The objective function could be formulated as

$$\theta = \arg \min_{\theta} \sum_{k=1}^K \frac{n_k}{n_a} L(T^k; \theta) \quad (1)$$

where  $T^k$  indicates the subcollection with  $n_k$  training samples at  $k$ th client and  $n_a$  is the sum of  $n_k$ .

Following the prior work [20], we define the CFL, which aims to optimize a unified model on sequentially independent tasks  $\{T_1, T_2, T_3, \dots\}$ . During the training phase of  $t$ th task, data belonging previous tasks will become unavailable, which fits the assumption in the real world. We denote the parameters of the unified model optimized on the previous tasks as  $\theta_{t-1}$ , and the new task as  $T_t = \cup_{k=1}^K T_t^k$ , where  $T_t^k$  refers to the  $k$ th data subcollection belonging to the new task. The optimization objective for  $t$  sequential tasks in CFL can be defined as

$$\theta_t = \arg \min_{\theta} \sum_{i=1}^t \sum_{k=1}^K \frac{n_k}{n_a} L(T_i^k; \theta). \quad (2)$$

The global model after sequential optimization with  $t$  tasks is expected to achieve favorable performance on all previous task. However, vanilla FL methods suffers CF without retraining on previous data. In the following section, we present our PI-Fed to mitigate this limitation with the cooperation of federated importance aggregation and soft-masking strategy.

#### IV. PI-FED FRAMEWORK

As discussed in Section I, CFL aims to achieve KT and overcome CF in FL for incremental task training. Following the previous work [17], the key for such a problem is to determines which parameters are more important to the old tasks and which parameters can be updated for the new tasks. However, if a neuron is totally masked, all parameters feeding to it are invalid, which leads to high-computational costs. PI-Fed soft-masks parameters for clients in task  $T_i$  based on their importance to previous tasks, which is computed at the end of each task training in a federated manner. With a more flexible masking strategy, PI-Fed clearly improve KT and overcome CF for FL. For reference and clarity, the definitions of symbols used in this section are listed in Table I.

##### A. Federated Importance Aggregation

1) *Importance Assessment*: Each time the PI-Fed framework completes one task training on  $T_i$  (the center server has broadcast the new global model weights), the importance of each parameter should be evaluated for soft masking (regularization) on later tasks. As shown in Fig. 2, the gradient on the historic losses could reflect how much changing it affects the model performance on old converged tasks. Hence, the historic losses should be first defined and formulated.

2) *Historic Losses*: We denote the loss for the previously learned tasks  $T_\tau$  as  $L^{i,\tau}$  ( $\forall \tau \in \mathbb{N}^+, \tau \leq i$ ). Note that current clients still hold data belonging to  $T_i$ , so  $L^{i,i}$  can be easily obtained by performing one more backward pass yet without updating parameters, as is illustrated in Fig. 3. However, obtaining the historic losses of  $T_\tau$  ( $\tau < i$ ) is a little bit tricky since there is no buffer of the old data for replay. According to [47], extreme logits distribution (lower logits sum) of the output layer indicates higher inference confidence. So we store copies of the classifier header after each task, and use the inference confidence [47] of the model with replaced headers on current data to represent  $L^{i,\tau}$  ( $\tau < i$ ). Then the unified formulation of  $L^{i,\tau}$  can be written as

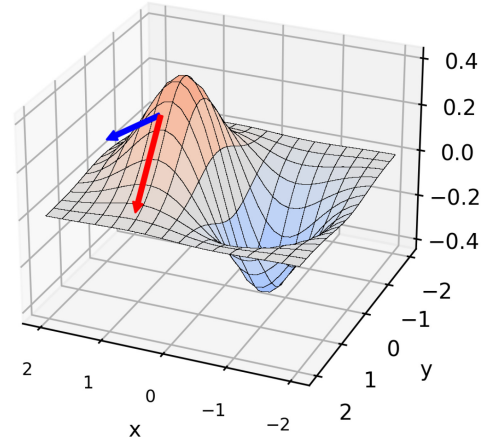
$$L^{i,\tau} = \begin{cases} L(f_\tau(X_k^i), Y_k^i; \theta) & (\tau = i) \\ \text{Sum}(f_\tau(X_k^i); \theta) & (\tau < i) \end{cases} \quad (3)$$

where  $f_\tau$  is the global model with the  $\tau$ th task head, and  $X_k^i$  and  $Y_k^i$  are the labeled data pairs at client  $k$  for training  $T_i$ .

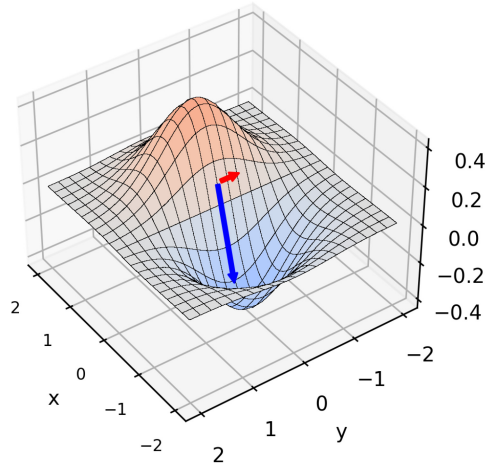
3) *Importance Aggregation*: Using the gradient value computed by  $L^{i,\tau}$ , client  $k$  computes the local normalized importance of the parameters  $\beta_k^{i,\tau}$

$$\text{Norm}(x) = \frac{x - \text{Mean}(x)}{\sqrt{\text{Var}(x)}} \quad (4)$$

$$\beta_k^{i,\tau} = \left| \tanh\left(\text{Norm}\left(\frac{\delta L^{i,\tau}}{\delta \theta_k}\right)\right) \right|. \quad (5)$$

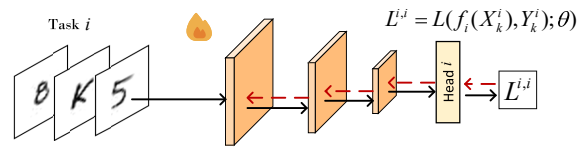


(a)

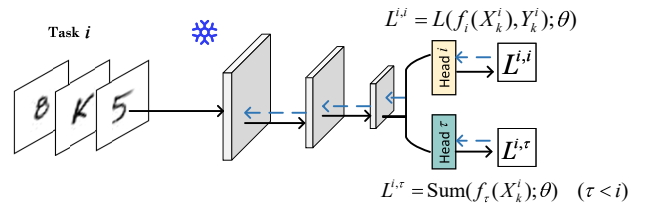


(b)

Fig. 2. Definition of parameter importance: The absolute value of gradient reflects how much changing it affects the final loss of old converged task. (a) Parameter Importance:  $X < Y$ . (b) Parameter Importance:  $X > Y$ .



1) Backpropagation for Update with Soft-masking



2) Backpropagation for Importance Computation

Fig. 3. Visualization of local training and importance computation.

Client  $k$  tasks the maximum importance value from different previous tasks are recognized as the locally updated importance for each parameter

$$\beta_k^i = \max(\beta_k^{i,1}, \dots, \beta_k^{i,i}). \quad (6)$$



Then all clients send the local importance mask to the center server and the server computes a normalized importance

$$\beta^i = \text{Norm} \left( \sum_{k=1}^K \beta_k^i \right). \quad (7)$$

Finally, the server stores the newly computed importance mask, which will be used to soft-mask the averaged gradient for future model aggregation. The soft-masking strategy will be described in detail in the next section.

### B. Federated Learning With Gradient Soft Masking

With the parameter importance  $\beta_{i-1}$  computed at the end of task  $T_{i-1}$ , we can use it as a soft mask to regularize the gradient when optimizing on task  $T_i$ , meaning that the import parameters is softly fixed to mitigate CF on old tasks and achieve KT with prior experience. Moreover, the soft masking is only performed in the gradient averaging stage at the server side, which retains high efficiency. Details about the soft masking progress are described as follows.

1) *Local Client Training*: For the training on the  $i$ th task, clients first compute the gradient with local data and then update their models independently. To differentiate the gradient accumulated during training from the gradient computed for parameter importance, we denote the gradient during training at client  $k$  in the local training phase as  $g_k^i$

$$g_k^i = \frac{\delta L^{i,i}}{\delta \theta_k}. \quad (8)$$

As shown in Fig. 1(b)-1, when the models converge on their local data, each client sends the accumulated gradient to the server for further aggregation.

2) *Gradient Averaging With Soft Masking*: After the server receives local accumulated gradient from all clients, it first averages them as the precalibrated global gradient as is shown in Fig. 1(b)-2

$$g^i = \frac{1}{K} \sum_{k=1}^K g_k^i. \quad (9)$$

Then the averaged gradient of each parameter is regularized with its normalized importance as a soft mask, which is introduced in Section IV-A

$$g_\star^i = \begin{cases} g^i, & \text{if } i = 1 \\ (1 - \beta^{i-1})g^i, & \text{otherwise} \end{cases} \quad (10)$$

and the global model weight  $w_i$  is updated according to the learning rate  $\eta$  and the calibrated gradient  $g_\star^i$

$$w'_i \leftarrow w_i - \eta g_\star^i. \quad (11)$$

Finally, as a new global checkpoint for the next round of training, the server broadcasts the weight  $w'_i$  to all clients.

### C. Summary of PI-Fed Framework

Fig. 1(a) and Algorithm 1 illustrate how PI-Fed computes the parameter importance and soft masking the gradient during FL. We now chronologically present the complete overview on the training process of PI-Fed.

### Algorithm 1 Chronological Training Process on Task $T_i$

---

```

1: # Function Definitions
2: function LOCALIMPORTANCE( $T_i, k$ )    ▷ Client  $k$  after task  $T_i$ 
3:   Unified loss function  $L^{i,\tau} \leftarrow$  Equation (3)
4:   Locally Computed Importance  $\beta_k^i \leftarrow$  Equation (4)(5)(6)
5: end function
6: function LOCALTRAIN( $T_i, k$ )    ▷ Client  $k$  during task  $T_i$ 
7:   Client  $k$  computes loss  $g_k^i \leftarrow$  Equation (8)
8: end function
9: # Clients (Local Training)
10: for  $k$  in range( $K$ ) do    ▷ Synchronized Procedures
11:   LOCALTRAIN( $T_i, k$ )
12:   Send Local Loss  $g_k^i$  to the Center Server
13: end for
14: # Server (Gradient Averaging and Soft Masking)
15: Average all local gradients for  $g^i \leftarrow$  Equation (9)
16: Soft masking  $g^i$  for calibrated  $g_\star^i \leftarrow$  Equation (10)
17: Broadcast the Updated Model Weight  $w'_i$  to all clients
18: # Clients (Importance Assessment)
19: for  $k$  in range( $K$ ) do    ▷ Synchronized Procedures
20:   LOCALIMPORTANCE( $T_i, k$ )
21:   Send Local Importance  $\beta_k^i$  to the Center Server
22: end for
23: # Server (Importance Aggregation)
24: Aggregated Importance Mask  $\beta^i \leftarrow$  Equation (7)
25: Store Newly Computed Importance Mask

```

---

- 1) *Local Training*: At the beginning of task  $T_i$ , all clients possess the same global model and unique private data. Each client  $k$  trains the model locally with labeled data pairs  $\langle X_k, Y_k \rangle$  to obtain gradient  $g_k^i$  as described in (8). After one epoch of training on the local data belonging to  $T_i$ , all the local gradients are sent to the center server as shown in Fig. 1(b)-1.
- 2) *Gradient Averaging and Soft Masking*: The center server first averages the local gradients to obtain precalibrated global gradient  $g^i$  according to (9). To alleviate CF and achieve KT, the averaged gradient is regularized with the soft mask  $\beta_{i-1}$  based on (10). Then, the server broadcasts the calibrated gradient to all clients as the new global model.
- 3) *Importance Assessment*: When completing the training on  $T_i$ , all clients evaluate the importance of each parameter to previous tasks according to (3)–(6). Then all clients send the local importance to the center sever.
- 4) *Importance Aggregation*: With importance values sent from each client, the center server computes a normalized importance as the new soft mask  $\beta_i$ , which will be used to calibrate aggregated gradient during task  $T_{i+1}$ .

### V. EXPERIMENTAL SETUP

For comparison, we opt to evaluate a battery of CL and FL methods on different incremental training tasks. Note that the classic CL baselines are not implemented in a federated

TABLE II  
STATISTICS ABOUT THE CFL DATA SETS USED FOR OUR EXPERIMENTS

| Dataset           | Size             | Tasks | Classes per task | #Train  | #Validation | #Test  | Task Type  |
|-------------------|------------------|-------|------------------|---------|-------------|--------|------------|
| CIFAR100- $n$     | $32 \times 32$   | $n$   | $100/n$          | 45,000  | 5,000       | 10,000 | Dissimilar |
| TinyImageNet- $n$ | $64 \times 64$   | $n$   | $200/n$          | 90,000  | 10,000      | 10,000 | Dissimilar |
| F-CelebA- $n$     | $178 \times 218$ | $n$   | 2                | $400n$  | $40n$       | $80n$  | Similar    |
| F-EMNIST- $n$     | $28 \times 28$   | $n$   | 62               | $3100n$ | $3100n$     | $620n$ | Similar    |

manner, all experiments on them are conducted with a single device. While all the FL baselines enable distributed training based on the communication among a center server and multiple clients. Our PI-Fed approach supports both the CL and distributed FL by adopting soft-masking strategy to calibrate the averaged gradient with parameter-level importance.

#### A. CFL Data Sets

As shown in Table II, the proposed PI-Fed and other baseline models are evaluated with four data sets: 1) CIFAR100- $n$ ; 2) TinyImageNet- $n$ ; 3) F-CelebA- $n$ ; and 4) F-EMNIST- $n$ . Following the experimental settings of the previous work [17], we use “- $n$ ” to represent that each data set is divided into  $n$  tasks ( $n$  takes 10 or 20). The former two data sets can be split into disjoint set of classes, while tasks created from the other two data sets have the same set of classes with different samples. Therefore, the tasks from the former data sets are referred as “dissimilar tasks,” and we regard the tasks created from the latter data sets as “similar data sets.” Two types of task collections could examine the ability of different CFL methods to solve CF and improve KT.

1) *CIFAR100- $n$  (C- $n$ ):* The CIFAR100 [48] data set consists of 60 000 images, divided into 100 classes. Each class contains 600 images, split into 500 for training and 100 for testing. The images are colored and of size  $32 \times 32$  pixels. In our experiments, the 100 different classes are grouped into 10 or 20 categories as the C-10 and C-20 tasks.

2) *TinyImageNet- $n$  (T- $n$ ):* The TinyImageNet [49] data set contains 100 000 images of 200 classes (500 for each class) downsized to  $64 \times 64$  colored images. Each class has 500 training images, 50 validation images and 50 test images. Respectively, T-10 and T-20 divide 200 classes into 10 and 20 disjoint sets.

3) *F-CelebA- $n$  (FC- $n$ ):* Federated CelebA [50] is a data set consisting of celebrities’ face images with annotated attributes. We only use CelebA with binary labels indicating whether the person in the image is smiling or not. Each set of images in  $n$  tasks consists of one celebrity.

4) *F-EMNIST- $n$  (FE- $n$ ):* The Federated Extended MNIST [51] is a data set that has 62 classes of handwritten digits with a  $28 \times 28$  format. Each task FE-10 or FE-20 contains 62 classes of symbols written by the same person.

#### B. Evaluation Metrics

For each CFL task, two evaluation metrics are reported.

##### 1) Latest Accuracy (LA):

$$LA(i) = \frac{1}{N_i} \sum_{j=1}^{N_i} \mathbb{1}[y_j = f(x_j)] \quad \forall x_j, y_j \in T_i. \quad (12)$$

##### 2) Averaged Accuracy (Avg):

$$Avg(i) = \frac{1}{i} \sum_{j=1}^i LA(j), i \in [1, n] \quad (13)$$

where  $T_i$  is the latest learned task by model  $f$ ,  $(x_i, y_i) \in T_i$  is a pair of data example and its groundtruth label, and  $N_i$  is the number of test samples in  $T_i$ . The LA represents the model’s convergence on the current task, while the Averaged Accuracy focuses on the model’s performance on all previously learned tasks.

#### C. Baseline Models

For comparison on CFL tasks defined in Section V-A, we reproduce and evaluate 13 baselines (which is illustrated in Table III). Ten of them are classic and recently advanced CL methods: 1) PGN [52]; 2) PathNet [53]; 3) HAT [54]; 4) CAT [55]; 5) SupSup [15]; 6) UCL [56]; 7) SI [57]; 8) WSN [16]; and 9) EWC [12]; 10) Besides, we refer to Naive CL as NCL, which learns each task without any mechanism to take care of CF. Note that the classic CL baselines are not implemented in a federated manner, all experiments on them are conducted on a single device.

Essentially, FL is a form of collaborative machine learning without centralizing training data. So we evaluate 3 fundamental FL algorithms on CFL tasks with additional requirement of dealing with CF and enhancing KT. Here, we make a brief review on them for reference (symbols used in this context are not related to previous definitions):

1) *FedAvg:* FedAvg [5] is the most popular FL algorithm for FL optimization due to its simple implementation, stateless nature, and privacy guarantees combined with secure aggregation

$$\begin{aligned} \forall k, w_{t+1}^k &\leftarrow w_t^k - \eta g_k \\ w_{t+1} &= \frac{1}{K} \sum_{k=1}^K w_{t+1}^k. \end{aligned} \quad (14)$$

In particular, FedAvg first locally performs SGD on  $K$  devices, the devices then communicate their model updates to a central server, where they are averaged.

TABLE III

ACCURACY EVALUATION WITH DIFFERENT TASK SETTINGS IN PERCENT. THE BEST MODELS ON EACH DATA SET ARE EMPHASIZED IN *Bold*, AND SECOND BEST RESULTS ARE UNDERLINED. CLASSIC CL METHODS ARE MARKED WITH ✗, WHILE THE FL BASELINES ARE MARKED WITH ✓

| Method        | Fed. | Dissimilar Tasks ↑ |             |             |             |             | Similar Tasks ↑ |             |             |             |             |
|---------------|------|--------------------|-------------|-------------|-------------|-------------|-----------------|-------------|-------------|-------------|-------------|
|               |      | C-10               | C-20        | T-10        | T-20        | Avg         | FC-10           | FC-20       | FE-10       | FE-20       | Avg         |
| NCL           | ✗    | 50.9               | 54.2        | 37.2        | 41.1        | 45.6        | 84.4            | 84.1        | 86.1        | 86.4        | 85.3        |
| PGN           | ✗    | 65.1               | 75.5        | 44.0        | 53.5        | 59.5        | 74.7            | 74.7        | 82.5        | 82.6        | 78.6        |
| PathNet       | ✗    | 69.1               | 75.5        | 46.0        | 51.9        | 60.6        | 79.3            | 80.5        | 84.3        | 84.5        | 82.1        |
| HAT           | ✗    | 62.8               | 71.8        | 45.5        | 51.7        | 59.9        | 79.0            | 81.9        | 83.8        | 84.6        | 82.3        |
| CAT           | ✗    | 64.2               | 73.9        | 43.7        | 50.9        | 58.2        | 82.9            | 82.9        | 82.9        | 84.1        | 83.2        |
| SupSup        | ✗    | 66.2               | 75.6        | 44.0        | 54.1        | 60.0        | 75.0            | 78.1        | 80.5        | 79.7        | 78.3        |
| UCL           | ✗    | 64.8               | 74.0        | 45.4        | 55.1        | 59.8        | 86.2            | 86.5        | 85.1        | 85.0        | 85.7        |
| SI            | ✗    | 62.9               | 70.3        | 45.9        | 52.6        | 57.9        | 86.4            | 86.8        | 87.6        | <u>87.9</u> | 87.2        |
| WSN           | ✗    | <b>69.3</b>        | <b>76.9</b> | 47.8        | 57.8        | 63.0        | 83.9            | 84.1        | 85.5        | 86.3        | 84.9        |
| SPG           | ✗    | 67.7               | 75.9        | <u>48.4</u> | <b>59.1</b> | <u>62.8</u> | 87.0            | 87.1        | <u>87.7</u> | 87.9        | 87.4        |
| EWC           | ✗    | 61.6               | 60.7        | 36.5        | 41.5        | 50.1        | 81.2            | 86.1        | 86.9        | 86.8        | 85.2        |
| FedAvg        | ✓    | 64.3               | 68.8        | 45.7        | 50.7        | 57.3        | 88.3            | 87.1        | 86.4        | 86.1        | 87.0        |
| FedNova       | ✓    | 22.6               | 25.0        | 11.6        | 11.3        | 17.6        | 86.6            | 85.7        | 80.7        | 68.3        | 80.3        |
| SCAFFOLD      | ✓    | 64.0               | 68.7        | 44.1        | 51.1        | 57.0        | <u>88.6</u>     | <u>87.5</u> | <b>89.4</b> | 87.2        | <u>88.2</u> |
| <b>PI-Fed</b> | ✓    | <u>68.6</u>        | <u>76.6</u> | <b>49.6</b> | <u>57.9</u> | <b>63.2</b> | <b>91.1</b>     | <b>88.8</b> | 87.1        | <b>90.8</b> | <b>89.5</b> |

2) *FedNova*: FedNova [58] further analyzes the convergence of federated heterogeneous optimization algorithms, and provides a normalized averaging method that eliminates objective inconsistency while preserving fast error convergence

$$w_{t+1} - w_t = -\tau_{\text{eff}} \sum_{i=1}^m \alpha_i \cdot \eta d_i^{(t)}$$

$$\text{which optimizes } \tilde{F}(x) = \sum_{i=1}^m \alpha_i F_i(x) \quad (15)$$

where  $d_i^{(t)}$  is the normalized gradient in line with different client optimizers,  $\alpha_i$  indicates aggregation weights which determines the surrogate objective function  $\tilde{F}(x) = \sum_{i=1}^m \alpha_i F_i(x)$ , and  $\tau_{\text{eff}}$  represents the effective number of steps which could scale up or scale down the effect of aggregated updates.

3) *SCAFFOLD*: SCAFFOLD [59] uses variates (variance reduction) to correct the ‘client-drift’ in its local updates

$$w_{t+1}^k \leftarrow w_t^k - \eta (g^k + c - c_k)$$

$$c'_k = c_k - c + \frac{1}{K\eta} (w_{\text{global}} - w_{t+1}^k) \quad (16)$$

where  $c$  and  $c_k$  are the server control variate and the control variate of the  $k$ th client, which satisfy  $c_k = c + \sum_{k=1}^K c_k$ . It is proved that SCAFFOLD requires significantly fewer communication rounds and is not affected by data heterogeneity.

TABLE IV  
DEFAULT HYPERPARAMETER SETTINGS OF  
PI-FED AND THREE FL BASELINES

| Hyper-param      | FedAvg    | FedNova   | SCAFFOLD  | PI-Fed    |
|------------------|-----------|-----------|-----------|-----------|
| Batch Size       | 64        | 64        | 64        | 64        |
| Optimizer        | SGD       | Momentum  | SGD       | SGD       |
| Learning Rate    | $1e^{-3}$ | $1e^{-3}$ | $1e^{-3}$ | $1e^{-4}$ |
| Global Epoch     | 20        | 20        | 20        | 20        |
| Client Number    | 28/54     | 28/54     | 28/54     | 28/54     |
| Client Iteration | 10        | 10        | 5         | 5         |

#### D. Implementation Details

For the classic CL methods, networks are locally trained with SGD by minimizing the cross-entropy loss. The batch size is set to be 64. Other hyperparameters, such as the dropout rate or each method’s specific hyperparameters, are searched based on TPE algorithm [60].

The hyperparameter settings for PI-Fed and other three FL baselines are listed in Table IV. The local client number according to the task number (28 for  $n = 10$  and 54 for  $n = 20$ ), the mini-batch size and the number of global epoch are set to be identical. The networks are also optimized with SGD except for FedNova, which uses Momentum-based optimizer for the default implementation. Based on our observation on the convergence speed, the learning rate and the local iteration number used for each baseline are slightly different. With predefined parameters, the experiments are conducted 5 times with different seeds, and the average accuracy is reported.

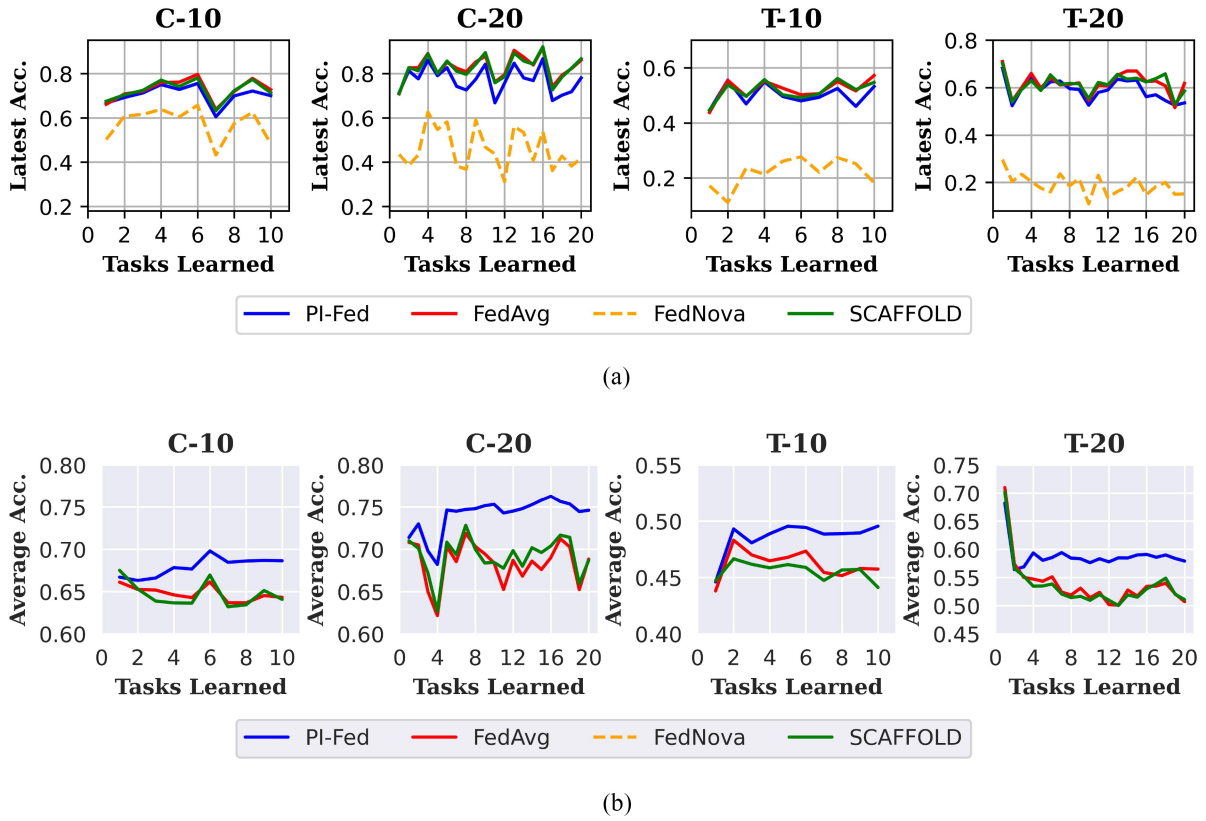


Fig. 4. Comparison between PI-Fed and other FL methods in the scenario of CFL. (a) *Evaluation on Dissimilar Tasks*: Accuracy on the latest learned task during CFL. (b) *Evaluation on Dissimilar Tasks*: Averaged accuracy over all learned tasks during CFL.

The three FL baselines (FedAvg, FedNova and SCAFFOLD) are reproduced based on the open-source library LEAF.<sup>2</sup> We adopt AlexNet [61] with different input size as the network backbones on each task. All distributed training and performance evaluation are simulated based on a system with an Intel Xeon Gold 6130 2.10-GHz processor and a NVIDIA RTX 4090 GPU with 24-G RAM.

## VI. MAIN RESULTS

Table III and Figs. 4 and 5 reports the results of the baselines and our proposed PI-Fed on 4 CFL tasks. Since classic CL methods are not originally implemented for the FL scenario, we only train a single model network with sequential tasks, which are marked with  $\times$  in table. FL approaches marked with  $\checkmark$  are simulated as a distributed system with server-client communication.

### A. Dissimilar Tasks

CIFAR100- $n$  (C- $n$ ) and TinyImageNet- $n$  (T- $n$ ) are two data sets that are divided in to  $n$  disjoint set of classes, where the main problem is to solve CF. Compared with classic CL methods, although WSN [16] slightly outperforms PI-Fed in C- $n$  and C-20, PI-Fed demonstrates superior performance on the average accuracy of dissimilar tasks. Specifically, methods based on gradient regularization are close to PI-Fed, whilst the representative methods EWC [12] and suffers heavily from CF

due to the unified regularization strategy. While PI-Fed adopts parameter-level regularization that could effectively alleviate the influence on old converged tasks during network update. Moreover, PI-Fed enables distributed training for FL which achieves strong privacy protection without significantly raising computational costs (details are discussed in Section VII-B).

Experimental results and visualization of sequential training process for FL baselines are illustrated in Table III and Fig. 4. Particularly, the LA metric shows that PI-Fed could effectively optimize the global model on new tasks just like other three FL baselines (FedAvg [5], FedNova [58], and SCAFFOLD [59]), which is not hindered by the soft-masking strategy. Moreover, the curves in Fig. 4 shows that the average accuracy of three FL baselines drops dramatically during the sequential training, which reflects the limited ability of solving CF. While PI-Fed with soft-masking could enable new task learning and meanwhile keep the forgetting slight, which markedly outperforms other FL baselines. On average, the accuracy for CFL on dissimilar task of applying PI-Fed increases 4.15% compared with traditional FL algorithms.

### B. Similar Tasks

In similar tasks created from F-CelebA- $n$  (FC- $n$ ) and F-EMINIST- $n$  (FE- $n$ ), classes are equally divided into each task, therefore the CF can hardly happen. KT is the essential problem we consider here. From Table III, PI-Fed achieves best in 3 out of 4 task cases. Among classic CL tasks, SPG, SI and NCL also perform well on similar CFL tasks, while

<sup>2</sup>A Benchmark for Federated Settings: <https://leaf.cmu.edu/>



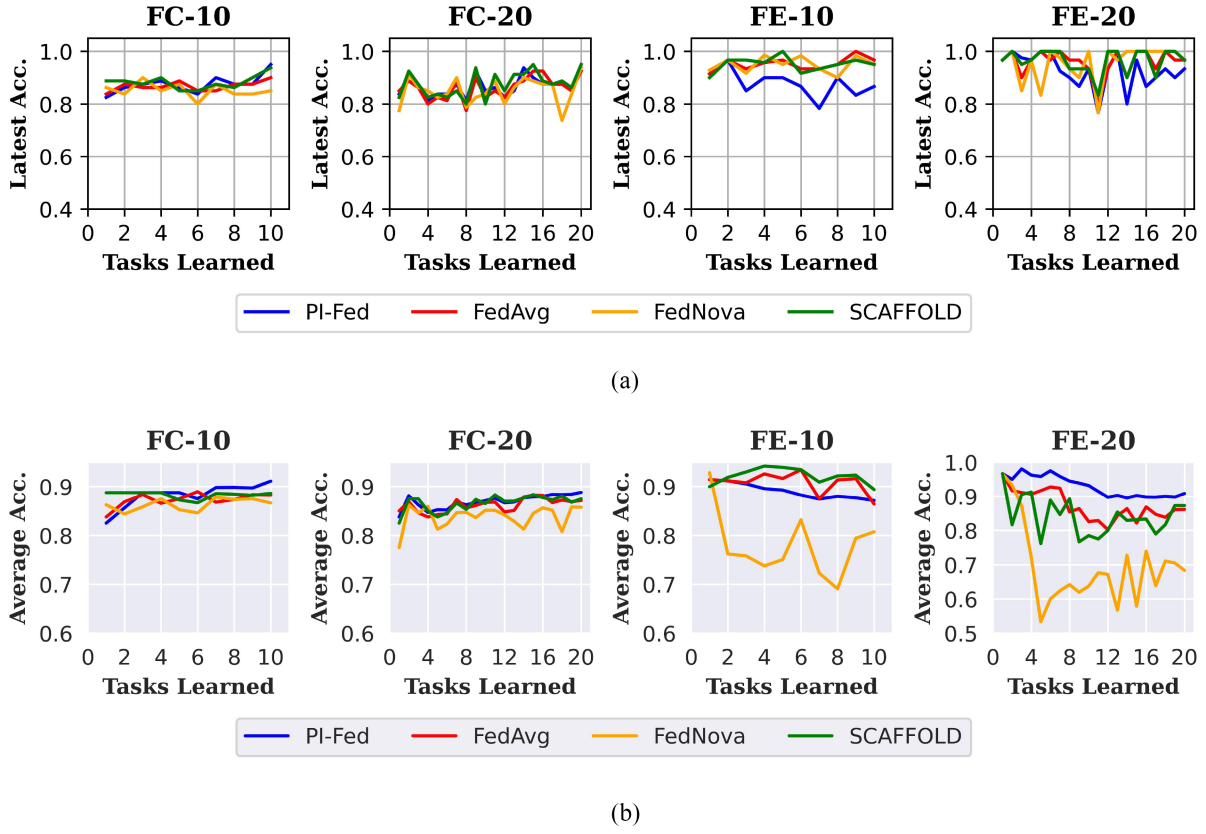


Fig. 5. Comparison between PI-Fed and other FL methods in the scenario of CFL. (a) *Evaluation on Similar Tasks*: Accuracy on the latest learned task during CFL. (b) *Evaluation on Similar Tasks*: Averaged accuracy over all learned tasks during CFL.

PGN, PathNet, HAT and CAT have lower average accuracy since their KT ability is limited due to strict regularization or just reuse previously learned parameters. Especially, SupSup reports lowest average accuracy since no KT strategy is integrated.

For the FL baselines, sequential experimental results are visualized in Fig. 5. We observe that all FL baselines could effectively optimize model on the latest task, but only PI-FED, FedAvg and SCAFFOLD perform backward KT (retain high accuracy on old tasks). FedNova under-performs other FL baselines due to overfitting on examples in the new task. As shown in Table III, our proposed PI-Fed achieves the best average in similar tasks, which indicates that PI-Fed has positive KT ability (high accuracy on both old and new tasks).

## VII. FURTHER ANALYSIS

In addition to the main reported results of our experiments, we further investigate two critical factors that could affect the performance of PI-Fed as a FL baseline, which also include discussion of the implication of the findings for future research into the implementation of PI-Fed in practice.

### A. Effect of Adaptive Optimizer

In this section, we investigate the effect of adaptive optimizers that may affect the performance of PI-Fed. The optimizer mostly seen in the FL literature is SGD [62], and it is also known that momentum and adaptive optimization methods

TABLE V  
ANALYSIS OF THE EFFECT OF DIFFERENT FIRST-ORDER OPTIMIZERS

| Optimizer  | Dissimilar <sub>LA</sub> | Dissimilar <sub>Avg</sub> | Similar <sub>LA</sub> | Similar <sub>Avg</sub> |
|------------|--------------------------|---------------------------|-----------------------|------------------------|
| RMSprop    | 35.4                     | 29.6                      | 58.5                  | 51.1                   |
| Adam       | 21.2                     | 18.2                      | 33.6                  | 29.8                   |
| AdaGrad    | 23.6                     | 20.0                      | 39.1                  | 32.9                   |
| Momentum   | 49.6                     | 46.8                      | 74.5                  | 68.6                   |
| <b>SGD</b> | <b>69.7</b>              | <b>63.2</b>               | <b>92.6</b>           | <b>89.2</b>            |

(such as Adam [46] and AdaGrad [44]) have become critical components for training neural networks, and can empirically improve the efficiency and generalization of SGD.

However, simply integrating adaptive optimizers to our proposed PI-Fed framework does not appear to be effective as expected. Based on PI-Fed, we compare canonical SGD with commonly used adaptive optimizers RMSprop [45], Adam [46], AdaGrad [44] and Momentum. Following the experimental settings described in Section V, the results of the latest and final average accuracy on dissimilar (C- $n$  and T- $n$ ) and similar tasks (FC- $n$  and FE- $n$ ) are reported in Table V. Note that the LA are averaged after each task. From Table V, we observe that PI-Fed with adaptive optimizers significantly underperform PI-Fed using SGD, which is contrary to expectations. A reasonable explanation is that the broadcast model calibrated at the center sever is inconsistent with the history gradient information stored in local optimizers (such as momentum). Taking the SGD with momentum as an example,

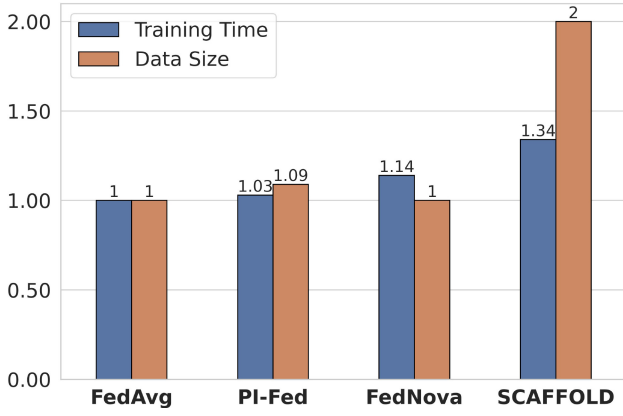


Fig. 6. Scaled comparison on communication cost of FL baselines.

the update of local weight at client  $k$  can be formulated as

$$m_t = \alpha m_{t-1} + (1 - \alpha) \left[ \frac{\delta L^{i,i}}{\delta \theta_k^t} \right]$$

$$\theta_k^{t+1} = \theta_k^t - \eta m_t \quad (17)$$

where  $\alpha$  controls the moving average of gradient momentum  $m_t$  and  $\eta$  represents the learning rate. In the case of PI-Fed, the gradients aggregated from each client to the center server are averaged and calibrated according to Algorithm 1, but the momentum  $m_t$  at each client is not updated which leads to the inconsistency and performance degradation. Empirically, Wang et al. [63] argued that even when the adaptive client optimizer states are reset to the default values at the beginning of each round, client adaptive methods for FL can be beneficial.

To address this issue, some advanced researches (framework based [64], server momentum based [65], client momentum based [66], [67] and lazy-update [68], [69] methods) have been recently proposed to enable adaptive optimization in FL. Therefore, exploring adaptive optimization for CFL could be recognized as a future work of our proposed PI-Fed, which could facilitate the application of PI-Fed in real-world scenarios.

### B. Efficiency Analysis

A major concern for the implementation of FL in practice is the communication cost. To analyze the communication efficiency, we evaluate and compare the training time consumption and the size of transmitted data of our proposed PI-Fed and other three FL baselines. Following the setting described in Section V, we record the statistics of CFL on CIFAR100-10 (C-10). Specifically, we regard the averaged time consumption and transmitted data size of each client per task of FedAvg as the units of measurement, the results of other FL methods are scaled accordingly and visualized in Fig. 6. Detailed observation is shown in Table VI.

As illustrated in Fig. 6 and Table VI, the canonical FedAvg needs the least training time and transmitted data size on average for each task in C-10. FedNova retains the size of transmitted data but has 14% more training time due to its optimization on the surrogate objective function. Since

TABLE VI  
DETAILED STATISTICS OF TIME CONSUMPTION AND  
TRANSMITTED DATA SIZE

| Consumption | FedAvg | PI-Fed | FedNova | SCAFFOLD |
|-------------|--------|--------|---------|----------|
| Time        | 37.6s  | 38.6s  | 43.0s   | 50.6s    |
| Data Size   | 498Mb  | 544Mb  | 498Mb   | 966Mb    |

the SCAFFOLD method uses local and global variates to correct the data heterogeneity in its local update, the size of transmitted data is doubled and the training time per task increase by 34% compared to FedAvg.

Whilst our proposed PI-Fed also needs to additionally transmit the soft-mask after each task and calibrate the gradient at the server for each round, the average training time and transmitted data size only have slight increment (3% and 9%). Collectively, this analysis demonstrates that our PI-Fed could effectively perform CFL with high efficiency.

## VIII. DISCUSSION

Our proposed PI-Fed is a novel framework designed for CFL, which provides a solution to the industrial needs for IoT, with various requirements on privacy preserving, distributed training and CL. Here, we envision three applicable scenarios in practice of our PI-Fed, including education, healthcare, and edge AI.

- 1) *Education*: After using FL to eliminate institutions' concerns on student privacy [70], PI-Fed could further open up possibilities for life-long knowledge tracing [71], educational analysis and courses recommendation [72].
- 2) *Healthcare*: FL is especially promising for smart healthcare since it allows numerous hospitals to participate on AI training whilst maintaining patient's privacy [73]. Without retraining and performance degradation, PI-Fed could flexibly increase the number of hospital clients participating tasks like mortality predication [74] and patients' activity recognition [75].
- 3) *Edge AI*: Training and inference of AI algorithms on edge devices enables the development of smart cities and smart factories [76], [77]. Under the premise of data security and privacy preserving, PI-Fed allows for efficient and continual update on networks running on mobile phones, IoT devices and sensor-equipped machines.

Although PI-Fed have demonstrated remarkable performance on CFL tasks, there are still aspects that can be improved for practical applications: 1) currently, PI-Fed only supports the canonical SGD optimizer and thus cannot benefit from adaptive optimization, which may lead to time overload on large data sets and 2) moreover, recent large language models (LLMs) has been reshaping the field of AI with powerful learning ability [78]. To enable CFL on LLMs, PI-Fed should integrate parameter-efficient tuning methods [79], such as prompt tuning and LoRA [80], otherwise the communication cost for gradient transmission could be unaffordable.

## IX. CONCLUSION AND FUTURE WORK

Recent years have seen explosive development of FL in IoT applications, which indicates a new need for CFL that enables long-term improvement of federated models. In this article, we propose PI-Fed, a novel framework designed for CFL based on gradient regularization, parameter-level importance aggregation and soft-masking strategy. Extensive experimental results on 4 data sets demonstrate that PI-Fed effectively achieve KT without forgetting, and significantly outperforms other FL baselines, which sets the stage for future research on CFL.

However, we acknowledge that PI-Fed can be further improved on some aspects to meet industrial needs. In the future, we plan to combine PI-Fed with adaptive optimizers which could accelerate convergence and improve generalization performance following recent works [64], [65], [69], and integrate parameter-efficient tuning methods [79] into PI-Fed to open the possibility of CFL on LLMs for the new AI era.

## REFERENCES

- [1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for smart cities," *IEEE Internet Things J.*, vol. 1, no. 1, pp. 22–32, Feb. 2014.
- [2] S. Wachter and B. Mittelstadt, "A right to reasonable inferences: Rethinking data protection law in the age of big data and AI," *Colum. Bus. L. Rev.*, vol. 7, no. 2, p. 494, 2019.
- [3] G. Zhang, B. Liu, T. Zhu, M. Ding, and W. Zhou, "PPFed: A privacy-preserving and personalized federated learning framework," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19380–19393, Jun. 2024.
- [4] J. Wu et al., "FedAPT: Joint adaptive parameter freezing and resource allocation for communication-efficient federated vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19520–19536, Jun. 2024.
- [5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [6] C. Xu, Y. Qu, Y. Xiang, and L. Gao, "Asynchronous federated learning on heterogeneous devices: A survey," 2021, *arXiv:2109.04269*.
- [7] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A survey on federated learning for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 1–24, Jan. 2022.
- [8] M. De Lange et al., "A continual learning survey: Defying forgetting in classification tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3366–3385, Jul. 2022.
- [9] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–17.
- [10] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with A-GEM," 2018, *arXiv:1812.00420*.
- [11] F. M. Castro, M. J. Marín-Jiménez, N. Guil, C. Schmid, and K. Alahari, "End-to-end incremental learning," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 233–248.
- [12] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [13] G. Saha, I. Garg, and K. Roy, "Gradient projection memory for continual learning," 2021, *arXiv:2103.09762*.
- [14] S. Kim, L. Noci, A. Orvieto, and T. Hofmann, "Achieving a better stability-plasticity trade-off via auxiliary networks in continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11930–11939.
- [15] M. Wortsman et al., "Supermasks in superposition," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 15173–15184.
- [16] H. Kang et al., "Forget-free continual learning with winning subnetworks," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 10734–10750.
- [17] T. Konishi, M. Kurokawa, C. Ono, Z. Ke, G. Kim, and B. Liu, "Parameter-level soft-masking for continual learning," 2023, *arXiv:2306.14775*.
- [18] N. Shoham et al., "Overcoming forgetting in federated learning on non-iid data," 2019, *arXiv:1910.07796*.
- [19] A. Usmanova, F. Portet, P. Lalanda, and G. Vega, "A distillation-based approach integrating continual learning and federated learning for pervasive services," 2021, *arXiv:2109.04197*.
- [20] Y. Ma, Z. Xie, J. Wang, K. Chen, and L. Shou, "Continual federated learning based on knowledge distillation," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, vol. 3, 2022, pp. 2182–2188.
- [21] L. Li, Y. Fan, M. Tse, and K.-Y. Lin, "A review of applications in federated learning," *Comput. Ind. Eng.*, vol. 149, Nov. 2020, Art. no. 106854.
- [22] S. H. Alsamhi et al., "Federated learning meets blockchain in decentralized data sharing: Healthcare use case," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19602–19615, Jun. 2024.
- [23] C.-H. Hu, Z. Chen, and E. G. Larsson, "Scheduling and aggregation design for asynchronous federated learning over wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 874–886, Apr. 2023.
- [24] W. Wu, L. He, W. Lin, R. Mao, C. Maple, and S. Jarvis, "SAFA: A semi-asynchronous protocol for fast federated learning with low overhead," *IEEE Trans. Comput.*, vol. 70, no. 5, pp. 655–668, May 2021.
- [25] Z. Chen, W. Liao, K. Hua, C. Lu, and W. Yu, "Towards asynchronous federated learning for heterogeneous edge-powered Internet of Things," *Digit. Commun. Netw.*, vol. 7, no. 3, pp. 317–326, 2021.
- [26] C. Xie, S. Koyejo, and I. Gupta, "Asynchronous federated optimization," 2019, *arXiv:1903.03934*.
- [27] Y. Chen, X. Sun, and Y. Jin, "Communication-efficient federated deep learning with layerwise asynchronous model update and temporally weighted aggregation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 10, pp. 4229–4238, Oct. 2020.
- [28] G. Shi, L. Li, J. Wang, W. Chen, K. Ye, and C. Xu, "HySync: Hybrid federated learning with effective synchronization," in *Proc. IEEE 22nd Int. Conf. High Perform. Comput. Commun.; IEEE 18th Int. Conf. Smart City; IEEE 6th Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, 2020, pp. 628–633.
- [29] M. Li, Y. Chen, Y. Wang, and Y. Pan, "Efficient asynchronous vertical federated learning via gradient prediction and double-end sparse compression," in *Proc. 16th Int. Conf. control, Autom., Robot. Vis. (ICARCV)*, 2020, pp. 291–296.
- [30] H.-S. Lee and J.-W. Lee, "Adaptive transmission scheduling in wireless networks for asynchronous federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3673–3687, Dec. 2021.
- [31] Z. Chai, Y. Chen, L. Zhao, Y. Cheng, and H. Rangwala, "FedAT: A communication-efficient federated learning method with asynchronous tiers under non-iid data," 2020, *arXiv:2010.05958v1*.
- [32] Y. Zhang et al., "CSAFL: A clustered semi-asynchronous federated learning framework," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–10.
- [33] Z. M. Fadlullah and N. Kato, "HCP: Heterogeneous computing platform for federated learning based collaborative content caching towards 6G networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 1, pp. 112–123, Jan.–Mar. 2022.
- [34] D. Huba et al., "Papaya: Practical, private, and scalable federated learning," in *Proc. Mach. Learn. Syst.*, vol. 4, 2022, pp. 814–832.
- [35] T. Zhang, L. Gao, S. Lee, M. Zhang, and S. Avestimehr, "TimelyFL: Heterogeneity-aware asynchronous federated learning with adaptive partial training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5064–5073.
- [36] D. Avdiukhin and S. Kasiviswanathan, "Federated learning under arbitrary communication patterns," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 425–435.
- [37] J. Nguyen et al., "Federated learning with buffered asynchronous aggregation," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2022, pp. 3581–3607.
- [38] H. Shin, J. K. Lee, J. Kim, and J. Kim, "Continual learning with deep generative replay," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–10.
- [39] W. Huang, M. Ye, and B. Du, "Learn from others and be yourself in heterogeneous federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10143–10153.
- [40] X. Yao and L. Sun, "Continual local training for better Initialization of federated models," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2020, pp. 1736–1740.
- [41] J. Yoon, W. Jeong, G. Lee, E. Yang, and S. J. Hwang, "Federated continual learning with weighted inter-client transfer," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12073–12086.
- [42] R. V. Kashyap, "A survey of deep learning optimizers-first and second order methods," 2022, *arXiv:2211.15596*.
- [43] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Comput. Math. Math. Phys.*, vol. 4, no. 5, pp. 1–17, 1964.

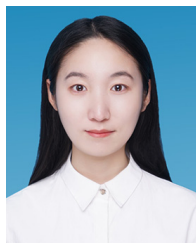


- [44] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *J. Mach. Learn. Res.*, vol. 12, no. 61, pp. 2121–2159, 2011.
- [45] G. Hinton, N. Srivastava, and K. Swersky, "Neural networks for machine learning lecture 6a overview of mini-batch gradient descent," *Cited on*, vol. 14, no. 8, p. 2, 2012.
- [46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [47] A. A. Taha, L. Hennig, and P. Knoth, "Confidence estimation of classification based on the distribution of the neural network output layer," 2022, *arXiv:2210.07745*.
- [48] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, 2009.
- [49] J. Wu, Q. Zhang, and G. Xu, *Tiny ImageNet Challenge*, Stanford University, Stanford, CA, USA, 2017.
- [50] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 3730–3738.
- [51] G. Cohen, S. Afshar, J. Tapson, and A. Van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. Int. Joint Conf. neural Netw. (IJCNN)*, 2017, pp. 2921–2926.
- [52] A. A. Rusu et al., "Progressive neural networks," 2016, *arXiv:1606.04671*.
- [53] C. Fernando et al., "PathNet: Evolution channels gradient descent in super neural networks," 2017, *arXiv:1701.08734*.
- [54] J. Serra, D. Suris, M. Miron, and A. Karatzoglou, "Overcoming catastrophic forgetting with hard attention to the task," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4548–4557.
- [55] Z. Ke, B. Liu, and X. Huang, "Continual learning of a mixed sequence of similar and dissimilar tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 18493–18504.
- [56] H. Ahn, S. Cha, D. Lee, and T. Moon, "Uncertainty-based continual learning with adaptive regularization," in *Proc. 33rd Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–11.
- [57] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.
- [58] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 7611–7623.
- [59] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [60] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Proc. 24th Int. Conf. Adv. Neural Inf. Process. Syst.*, vol. 24, 2011, pp. 2546–2554.
- [61] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [62] H. Robbins and S. Monro, "A stochastic approximation method," *Ann. Math. Statist.*, vol. 22, no. 3, pp. 400–407, 1951.
- [63] J. Wang, Z. Xu, Z. Garrett, Z. Charles, L. Liu, and G. Joshi, "Local adaptivity in federated learning: Convergence and consistency," 2021, *arXiv:2106.02305*.
- [64] S. Reddi et al., "Adaptive federated optimization," 2020, *arXiv:2003.00295*.
- [65] J. Wang, V. Tantia, N. Ballas, and M. Rabbat, "SlowMo: Improving communication-efficient distributed SGD with slow momentum," 2019, *arXiv:1910.00643*.
- [66] H. Yu, R. Jin, and S. Yang, "On the linear speedup analysis of communication efficient momentum SGD for distributed non-convex optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7184–7193.
- [67] H. Yuan and T. Ma, "Federated accelerated stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 5332–5344.
- [68] C. Xie, O. Koyejo, I. Gupta, and H. Lin, "Local AdaAlter: Communication-efficient stochastic gradient descent with adaptive learning rates," 2019, *arXiv:1911.09030*.
- [69] S. P. Karimireddy et al., "Mime: Mimicking centralized stochastic algorithms in federated learning," 2020, *arXiv:2008.03606*.
- [70] S. Guo, D. Zeng, and S. Dong, "Pedagogical data analysis via federated learning toward education 4.0," *Am. J. Educ. Inf. Technol.*, vol. 10, no. 2, pp. 56–65, 2020.
- [71] Q. Liu, S. Shen, Z. Huang, E. Chen, and Y. Zheng, "A survey of knowledge tracing," 2021, *arXiv:2105.15106*.
- [72] Y. Qin, M. Li, and J. Zhu, "Privacy-preserving federated learning framework in multimedia courses recommendation," *Wireless Netw.*, vol. 29, no. 4, pp. 1535–1544, 2023.
- [73] A. Rahman et al., "Federated learning-based AI approaches in smart healthcare: Concepts, taxonomies, challenges and open issues," *Clust. Comput.*, vol. 26, no. 4, pp. 2271–2311, 2023.
- [74] Y. Kim, J. Sun, H. Yu, and X. Jiang, "Federated tensor factorization for computational phenotyping," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2017, pp. 887–895.
- [75] Y. Chen, X. Qin, J. Wang, C. Yu, and W. Gao, "FedHealth: A federated transfer learning framework for wearable healthcare," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 83–93, Jul./Aug. 2020.
- [76] M. Xu et al., "Unleashing the power of edge-cloud generative AI in mobile networks: A survey of AIGC services," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1127–1170, 2nd Quart., 2024.
- [77] R. Singh and S. S. Gill, "Edge AI: A survey," *Internet Things Cyber-Phys. Syst.*, vol. 3, pp. 71–92, Mar. 2023.
- [78] T. Brown et al., "Language models are few-shot learners," in *Proc. 34th Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [79] Z. Han, C. Gao, J. Liu, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," 2024, *arXiv:2403.14608*.
- [80] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.



**Lang Yu** (Graduate Student Member, IEEE) received the B.E. degree from the Department of Information Engineering, Zhengzhou University, Zhengzhou, China, in 2019, and the M.E. degree from the Department of Software Engineering, Henan University, Kaifeng, China, in 2022. He is currently pursuing the Ph.D. degree in computer science and technology with East China Normal University, Shanghai, China.

His research interests include on federated learning, machine learning, and deep learning.



**Lina Ge** received the B.S. and M.E. degrees from the Department of Software Engineering, Henan University, Kaifeng China, in 2019 and 2022, respectively.

She was a Senior Engineer for Cloud Storage with H3C Technologies Co., Hangzhou, China, from 2022 to 2023. She is currently a Lecturer with Henan Finance University, Zhengzhou, China. Her research interests include federated learning and cloud computing.



**Guanghui Wang** (Member, IEEE) received the Ph.D. degree in information networks from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2019.

He is currently an Associate Professor with the School of Software, Henan University, Kaifeng, China. He was a visiting research student with the Department of Computer Science, University of Victoria, Victoria, BC, Canada, from 2016 to 2018. His current research interests include blockchain, federated learning, security and privacy preservation, and Internet of Things.



**Jianghao Yin** received the B.E. degree from Hubei University of Technology, Wuhan, China, in 2023. He is currently pursuing the M.E. degree in computer science and technology with East China Normal University, Shanghai, China.

His research focuses on multimodal learning.





**Qin Chen** received the B.S. and Ph.D. degrees in computer science from East China Normal University, Shanghai, China, in 2013 and 2018, respectively.

She was an International Visiting Research Trainee with the Lab of Information Retrieval and Knowledge Management, York University, North York, ON, Canada, from 2017 to 2018. She is currently an Associate Professor with the School of Computer Science and Technology, East China Normal University. She has published more than 40

referred papers in top tier journals (such as the IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, the *Information Sciences*, and the *Knowledge-Based Systems*) and conferences (such as ACM SIGIR, ACM CIKM, AAAI, and EMNLP). Her research interests include information retrieval, natural language processing, and deep learning applied to multimedia and education.



**Jie Zhou** received the Ph.D. degree in computer science from East China Normal University, Shanghai, China, in 2021.

He is currently an Associate Professor with the School of Computer Science and Technology, East China Normal University. He has authored or co-authored more than 40 refereed papers in top-tier conferences and journal, such as AAAI, SIGIR, IJCAI, ACL, and *Information Sciences*. His research interests include sentiment analysis and natural language processing.



**Liang He** received the B.S. degree in computer science and technology, the M.S. degree in multimedia technology, and the Ph.D. degree in system analysis and integration from East China Normal University, Shanghai, China, in 1995, 1997, and 2000, respectively.

He is currently a Professor and the Director of the Department of Computer Science and Technology, East China Normal University. He has received more than ten patents, and published two monographs and more than 70 refereed papers in national and

international journals and conference proceedings. His research interests include knowledge processing, user behavior analysis, and context-aware computing.

Prof. He was the recipient of the Shanghai Science and Technology Progress Award for five times, the First Prize in 2013, and the Second Prize in 2015. He was also the recipient of the Star of the Talent in Shanghai. He is also a Council Member of the Shanghai Computer Society, a member of the Academic Committee, the Director of the Technical Committee of Shanghai Engineering Research Center of Intelligent Service Robot, and a Technology Foresight Expert of the Shanghai Science and Technology in focus areas.